

● 오라클 4일차

- overloaing - 함수이름은 같고 매개변수의 타입과 갯수가 다르도록 구현
 - to_char - date를 char형으로 변환하는 함수
 - to_char(날짜,'날짜모델요소')
 - to_char(숫자,'숫자모델요소')
 - /* 문제 22 오늘 날짜를 출력해주세요
 - 문제 23 사원의 employees(사원)테이블에 있는 last_name, hire_date 및 근무 6개월 후
 - 첫 번째 월요일에 해당하는 급여 협상 날짜를 표시합니다. 열 레이블을 REVIEW로 지정
 - 날짜는 월요일 , the Second 4, 2007과 유사한 형식으로 지정 */
 - SELECT
 - TO_CHAR(SYSDATE, 'fmyyyy"년" mm"월" dd"일" DAY', 'NLS_DATE_LANGUAGE=KOREAN')
 - FROM dual; -- 선행되는 제로 03/06 같이 앞에 보이는 걸 지우려면 fm 활용 fmmm << 3/6 출력
 - -- dl 이라는 함수 사용시 쉽게 출력 가능 to_char(sysdate,'dl')
 - -- ds 도 있음
 - SELECT
 - last_name,
 - hire_date,
 - TO_CHAR(
 - NEXT_DAY(ADD_MONTHS(hire_date, 6), '월요일'),
 - 'Day, "the" Ddthsp fmmm, YYYY',
 - 'NLS_DATE_LANGUAGE=KOREAN'
 -) AS REVIEW
 - FROM hr.employees;
 - desc hr.employees
 - -- 숫자에 대한 단위 표시
 - select
 - employee_id,
 - salary,
 - to_char(salary,'999,999') salary_1,
 - to_char(salary,'000,999') salary_2
 - from hr.employees;
 - select
 - employee_id,
 - salary,
 - to_char(salary,'999,999.99') salary_1,
 - to_char(salary,'\$999,999') salary_2,
 - to_char(salary,'l999g999d00') salary_3

- from hr.employees;
- -- , 자리에 g . 자리에 d 로 표현이 가능
-
- alter session set nls_language = american;
- alter session set nls_territory = america;
-
- select
- employee_id,
- salary,
- to_char(sysdate,'month mon day dy')day,
- to_char(salary,'l999g999d00') salary
- from hr.employees;
-
- alter session set nls_language = french;
- alter session set nls_territory = france;
-
- select
- employee_id,
- salary,
- to_char(sysdate,'month mon day dy')day,
- to_char(salary,'l999g999d00') salary
- from hr.employees;
-
- alter session set nls_language = korean;
- alter session set nls_territory = korea;
-
- select
- employee_id,
- salary,
- to_char(sysdate,'month mon day dy')day,
- to_char(salary,'l999g999d00') salary
- from hr.employees;
-
- select
- -1000,
- to_char(-1000,'9999pr'),
- to_char(-1000,'s9999')
- from dual;
- -- <> 로 표현해야될 경우 pr 사용
- -- 표현이 필요할 경우 s
-
- select
- last_name,
- salary,
- to_char(salary) sal_new,
- trim(to_char(salary,'99999')) sal
- from hr.employees;
- -- trim 제거 필요시 사용

-
- select
- 1+1,
- '1'+1, -- 암시적 형변환
- to_number('1')+1 -- 명시적 형변환
- from dual;
-
- to_number - 숫자문자(char)를 수치형(number) 자료로 형변환 함수
 - 문자숫자, 숫자모델요소(생략가능)
 - /* 문제 24 짝수달에 입사한 직원들의 정보를 출력
 - 문제 25 2006년도에 홀수달에 입사한 직원들의 정보 출력 */
 - select *
 - from hr.employees
 - where mod(to_number(to_char(hire_date, 'mm')),2) = 0;
 - select *
 - from hr.employees
 - where to_char(hire_date, 'yyyy') = '2006' and
 - - 어떤 환경의 날짜에서든 정보를 받고자 할 경우 to_date 함수 사용
 - to_date('날짜', '날짜요소' Ex ['2002-01-01', 'yyyy-mm-dd'])
 - where hire_date BETWEEN to_date('2006-01-01','yyyy-mm-dd') AND to_date('2007-01-01', 'yyyy-mm-dd');
 - 기준 컬럼이 변경되기 때문에 비교형태는 기준 컬럼에 맞게 해야 된다.
 - mod(to_number(to_char(hire_date, 'mm')),2) = 1;
 - to_date - 문자날짜 (char)를 날짜 (date) 형으로 변환
- NULL - 사용할 수 없거나 할당되지 않거나 알 수 없거나 적용할 수 없는 값
 - nvl - null 값을 실제 값으로 리턴하는 함수
 - 입력되는 인수값들의 데이터 타입이 일치해야 한다.
 - nvl(컬럼, 실제값)
 - nvl2(exp1, exp2, exp3)
 - exp1 null 아니면 exp2를 수행 exp1이 null이면 exp3 실행
 - exp2, exp3의 데이터타입이 일치해야한다.
 - SELECT
 - employee_id,
 - salary,
 - commission_pct,
 - salary * 12 + salary * 12 * commission_pct AS annual_salary,
 - salary * 12 AS annual_salary_1,
 - salary * 12 + salary * 12 * NVL(commission_pct, 0) AS annual_salary_2,
 - NVL2(commission_pct,salary * 12 + salary * 12 * commission_pct, salary * 12) AS annual_salary_3
 - FROM hr.employees;
 - — coalesce (exp1,exp2,exp3, ...expn)
 - exp1이 null이면 exp2 수행, exp2가 null이면 exp3 수행, exp3가 null이면 ... expn 수행 즉 null이 발생하지 않을 때 까지 수행
 - SELECT

- employee_id,
- salary,
- commission_pct,
- coalesce(salary * 12 + salary * 12 * commission_pct, salary * 12, 0)
AS annual_salary
- FROM hr.employees;

○ null if - 두 표현식을 비교해서 값이 같으면 null을 리턴 값이 틀리면 exp1을 리턴

- SELECT
- employee_id,
- length(last_name),
- length(first_name),
- nullif(length(last_name),length(first_name))
- FROM hr.employees;

○ 조건 제어문

- SQL에서 IF(PL/SQL)을 사용해서 조건 제어문을 수행할 수 없다.
- DECODE 함수, CASE 표현식

■ PL / SQL

- If 기준값 = 비교값1 THEN
 - 참값
- else if 기준값 = 비교값 2 THEN
 - 참값
- else if 기준값 = 비교값 3 THEN
 - 참값
- else
 - 기본값
- end if;
- - decode 함수는 기준값과 비교값을 같다 비교연산자를 사용
- decode(기준값, 비교값1, 참값1,
비교값2, 참값2,
비교값3, 참값3,
...
기본값)
- select
- employee_id,
- salary,
- job_id,
- decode(job_id,
- 'it_prog', salary * 1.1,
- 'st_clerk', salary * 1.2,
- 'sa_rep', salary * 1.3,
- salary) as revised_salary
- from hr.employees;
-

- -- case 표현식 : 기준값과 비교값의 대해서 모든 비교 연산자를 사용할 수 있다.
- 기준값 = 비교값
- case 기준값
 - when 비교값 1 then 참값1
 - when 비교값 2 then 참값2
 - when 비교값 3 then 참값3
 -
 - else 기본값
- end

비교연산자 (= , > , >= , < , <= , <> , != , ^= , in, between, like, is null, is not null)

논리연산자 (not, and, or)

- select
- employee_id,
- salary,
- job_id,
- case job_id
- when 'it_prog' then salary * 1.1
- when 'st_clerk' then salary * 1.2
- when 'sa_rep' then salary * 1.3
- else salary
- end as revised_salary
- from hr.employees;
-
- select
- employee_id,
- salary,
- job_id,
- case
- when job_id = 'it_prog' then salary * 1.1
- when job_id = 'st_clerk' then salary * 1.2
- when job_id = 'sa_rep' then salary * 1.3
- else salary
- end as revised_salary
- from hr.employees;
-
- /* 문제 26 사원들의 급여기준으로 추력
- ~ 4999 : low
- 5000 ~ 9999 : medium
- 10000 ~ 19999 : good
- 20000 ~ : excellent
- */
-
- select
- employee_id,
- salary,

- case
- when salary <= 4999 then 'low'
- when salary <= 9999 then 'medium'
- when salary <= 19999 then 'good'
- else 'excellent'
- end as selery_grand'

- from hr.employees;

- decode 함수, case 표현식에 null check 방법

decode 함수는 null 키워드를 사용해서 null check

case 표현식은 null 연산자를 null check

- select
- employee_id,
- salary,
- commission_pct,
- decode(commission_pct, null, salary * 12, salary * 12 + salary * 12 * commission_pct) annual_salary
- from hr.employees;

- select
- employee_id,
- salary,
- commission_pct,
- case
- when commission_pct is null then salary * 12
- else salary * 12 + salary * 12 * commission_pct
- end as annual_salary
- from hr.employees;
- select

- employee_id,
- salary,
- commission_pct,
- case
- when commission_pct is not null then salary * 12 + salary * 12 * commission_pct
- else salary * 20
- end as annual_salary
- from hr.employees;

- yy - 현재 년도의 세기를 반영

- rr - 2000년 표기법을 자동화로 변경

- select
- to_char(to_date('95-10-27','yy-mm-dd'),'yyyy-mm-dd') date_1,
- to_char(to_date('95-10-27','rr-mm-dd'),'yyyy-mm-dd')date_2
- from dual;

- 데이터 입력 연도 (년도)

	0 ~ 49	50 ~ 99
○ 현재연도		
○ 0 ~ 49	반환날짜는 현재세기 반영	반환날짜는 이전 세기를 반영
○ 50 ~ 99	반환날짜는 이후세기를 반영	반환날짜는 현재 세기를 반영
○ 현재연도	데이터입력날짜	YY RR
○ 1994	95-10-27	1995 1995
○ 1994	17-10-27	1994 2017
○ 2001	17-10-27	2017 2017
○ 2048	52-10-27	2052 1952
○ 2051	47-10-27	2047 2147

- 그룹함수 - 여러 행 당 하나의 결과를 반환하는 함수
 - 기본적으로 그룹 함수는 NULL을 포함하지 않지만 count(*)의 경우 NULL을 포함 시킨다.
 - 그룹함수에 입력하는 값이 숫자형만 입력해야하는 그룹함수는 : sum, avg, median, variance, stddev
 - 그룹함수에 입력하는 값이 모든 데이터타입이 가능한 그룹함수는 : max, min, count
 - sum, avg, median, variance, stddev, max, min, count
 - count - 행의 수를 구하는 함수
 - null을 포함한 수
 - select
 - count(*)
 - from hr.employees;
 - distinct - 중복을 제거한
 - select
 - count(distinct department_id)
 - from hr.employees;
 - sum - 합
 - select
 - sum(salary)
 - from hr.employees
 - where department_id = 50;
 - avg - 평균
 - select
 - avg(salary)
 - from hr.employees
 - where department_id = 50;
 - variance - 분산
 - select
 - avg(salary)
 - variance(salary)
 - from hr.employees;
 - stddev - 표준편차
 - select

- avg(salary)
- variance(salary)
- stddev(salary)
- from hr.employees;
- max - 최대값 , min - 최소값
 - select
 - max(salary)
 - min(salary)
 - max(salary) - min(salary)
 - from hr.employees;